Teach Me Skills

GOF Patterns и Best Practices автоматизации

Что такое pattern?

Design Pattern - это стандартный подход к решению определенной проблемы

Не существует такого понятия как «хороший дизайн-паттерн», «плохой дизайн-паттерн». Сам термин «дизайн-паттерн» был придуман в качестве формулировки проблемы и предложенного решения. Поэтому для каждого встреченного вами паттерна – в разработке, в тестировании, еще в чем-то, если вы не разделяете проблему, для которой он был придуман – это не значит, что он плохой или «вышел из моды», просто он для вас не подходит. Если ваша проблема каким-то образом совпадает и накладывается на этот дизайн-паттерн, значит, вам стоит его рассмотреть.

teachmeskills.by

Design-patterns в автоматизации

1. Page Object
2. Page Factory
3. Fluent/Chain of invocations
4. Loadable Page/Component

teachmeskills.by

Design-patterns в автоматизации

5 Value Object/DTO

6 Page Element/Wrappers

7 Builder/Factory

teachmeskills.by

Page Object

Проблемы:

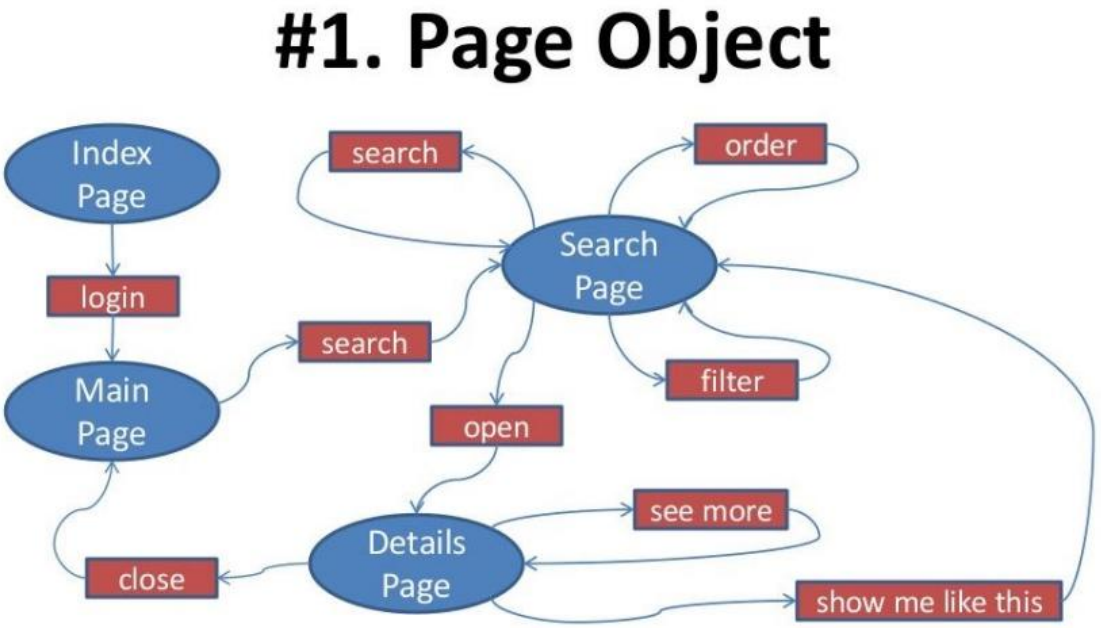
1. Код находится в одном классе, что затрудняет совместную работу
2. Список доступных действий в тесте слишком широкий
3. Код тестов нечитабелен

Решение:

1. Каждая страница/модалка/порой даже некая область на странице - выносится в отдельный класс. В данном классе только локаторы и методы для взаимодействия

teachmeskills.by

Структура



teachmeskills.by

Page Factory

Проблемы:

1. Повторяющиеся записи по поиску элементов внутри Page Objects

Решение:

1. Инициализация ВСЕХ элементов на странице происходит однократно с помощью аннотаций @FindBy

Page Factory

```
public class BlaBlaPage {
    WebDriver driver;
    @FindBy(name="uid")
    WebElement user99GuruName;
    @FindBy(name="password")
    WebElement password99Guru;
    @FindBy(className="barone")
    WebElement titleText;
    @FindBy(name="btnLogin")
    WebElement login;

    public BlaBlaPage(WebDriver driver) {
        this.driver = driver;
        //This initElements method will create a proxy class,
        // which will intercept accesses to web element.
        PageFactory.initElements(driver, this);
    }
}
```

teachmeskills.by

Page Factory - особенности

1. Инициализация элементов через аннотации **@FindBy** (поиск элемента по локатору), **@FindBys** (поиск элементов по локаторам с условием И, возвращает коллекцию), **@FindAll** (поиск элементов по условию ИЛИ, возвращает коллекцию)

2. Сама инициализация по факту происходит после вызова метода **PageFactory.initElements(driver, this);**
При этом создается специальный прокси-объект, который перехватывает доступы к FindBy полям. Он вызывает findElement и возвращает найденный элемент.

teachmeskills.by

Fluent/Chain of Invocations

Проблемы:

1. При вызове методов из Page Object не всегда ясно какой за каким следует использовать, а также приходится писать дублирующийся вызов переменной

Решение:

1. После вызова метода - возвращать тот же либо другой объект (если происходит переход на новую страницу) как возвращаемый параметр метода

teachmeskills.by

Как выглядит после имплементации

#2. Fluent/Chain of invocations

```
@Test
public void userNameSuggestions() {
    accountsPage.typeFirstLetterOfName("j");
    accountsPage.waitForSuggestions();
    accountsPage.selectFirstSuggestedName();

    Assert.assertEquals("John", accountsPage.getUserName());
}
```

```
@Test
public void userNameSuggestions() {
    accountsPage.typeFirstLetterOfName("j")
            .waitForSuggestions()
            .selectFirstSuggestedName();

    Assert.assertEquals("John", accountsPage.getUserName());
}
```

teachmeskills.by

Пример использования

```
public void nextPage() {
   new ChromeDriver().findElement(By.id("next-page-locator")).click();
}
```

```
public NextPageObject goToNextPage() {
   new ChromeDriver().findElement(By.id("next-page-locator")).click();
   return new NextPageObject();
}
```

teachmeskills.by

Loadable Page/Component

Проблемы:

1. После загрузки страницы - часть элементов может быть динамическими и на самом деле не вся страница загрузилась
2. Падения тестов в середине могут быть вызваны проблемой с недозагрузкой страниц

Решение:

1. Метод для проверки, что страница действительно загрузилась, который будет вызываться прежде чем будут выполняться некие действия

teachmeskills.by

Пример использования

```
public abstract class BasePage {
   public abstract BasePage isPageOpened();
   public abstract BasePage openPage();
}
```

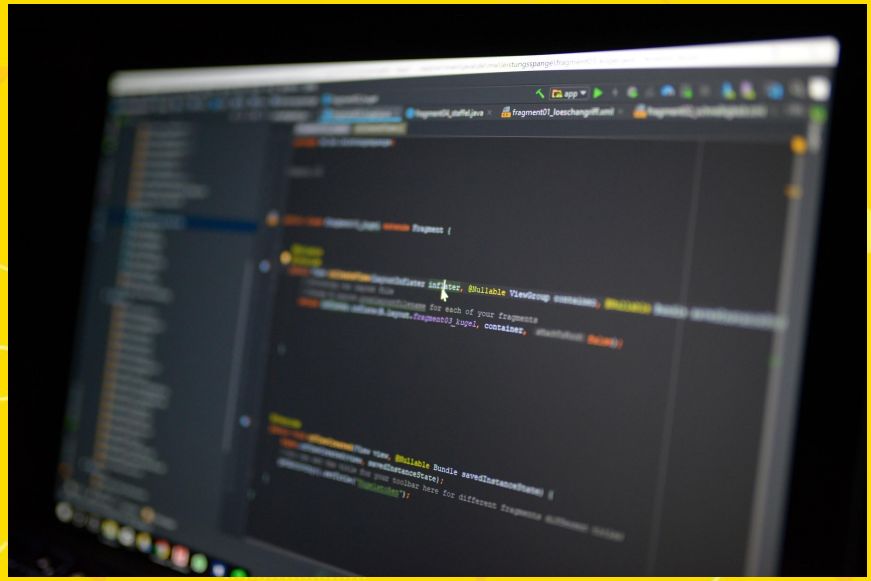
```
public class ParticularPage extends BasePage {

   @Override
   public BasePage isPageOpened() {
      //ОЖИДАНИЕ ПОКА НА СТРАНИЦЕ НЕ ОТОБРАЗИТСЯ
ПОСЛЕДНИЙ ЭЛЕМЕНТ
      return this;
   }

   @Override
   public BasePage openPage() {
      //МЕТОД ДЛЯ ОТКРЫТИЯ КОНКРЕТНОЙ СТРАНИЦЫ
      return this;
   }
}
```

teachmeskills.by

Value Object



Проблемы:

1. Некоторые методы принимают СЛИШКОМ МНОГО входных параметров одинакового типа, очередность их неясна
2. Если в метод необходимо добавить дополнительный параметр - обновлять придется все места, где он использовался

Решение:

1. Если некоторые параметры объединены в логический объект - создавать для него отдельный класс, содержащий все поля. Методы работают уже с объектами как входными параметрами

teachmeskills.by

Пример использования

#7. Value Object

```
public void registerUser(String name, String surname, int age,
                         Set<Role> roles, boolean paid) {
    enter(name, into( path: "name"));
    enter(surname, into( path: "surname"));
    enter( text: "" + age, into( path: "age"));
    chooseRoles(roles, paid);
}
```

```
public void registerUser(User user) {
    enter(user.getName(), into( path: "name"));
    enter(user.getSurname(), into( path: "surname"));
    enter(user.getAgeAsText(), into( path: "age"));
    chooseRoles(user.getRoles(), user.isPaid());
}
```

teachmeskills.by